

The *Attention* Mechanism

A comprehensive technical examination of attention in deep learning — from its theoretical foundations and mathematical formulation to multi-head architectures, transformer integration, and modern variants.



AUTHOR

Uditya Narayan Tiwari

DOMAIN

Deep Learning · NLP · Computer Vision

YEAR

2026

TABLE OF CONTENTS

01	Introduction & Motivation	§ 1
02	Historical Background	§ 2
03	Mathematical Foundations	§ 3
04	Types of Attention Mechanisms	§ 4
05	Scaled Dot-Product Attention	§ 5
06	Multi-Head Attention	§ 6
07	Self-Attention & Cross-Attention	§ 7
08	Attention in the Transformer Architecture	§ 8
09	Positional Encoding	§ 9
10	Advanced Variants	§ 10
11	Complexity & Efficiency	§ 11
12	Applications	§ 12
13	Implementation	§ 13

Introduction & Motivation

The **attention mechanism** is perhaps the single most important innovation in the history of modern deep learning. Introduced in the context of neural machine translation, it fundamentally solved a long-standing bottleneck in sequence modelling — the inability of fixed-length context vectors to faithfully represent arbitrarily long sequences — and in doing so, opened the door to the Transformer architecture that now underpins nearly every state-of-the-art model in natural language processing, computer vision, and beyond.

Before attention, encoder-decoder networks had to compress an entire source sentence into a single hidden state of fixed dimensionality. This created a severe information bottleneck: long sentences suffered catastrophically because early tokens were effectively forgotten by the time the decoder generated its final outputs. Bahdanau et al. (2015) proposed a radical alternative: rather than forcing the decoder to read from a single summary vector, allow it to *attend* to all encoder hidden states simultaneously, weighting each by its relevance to the current decoding step.

CORE INTUITION

Attention is a learnable, differentiable, soft-lookup mechanism. Given a **query** (what we're looking for), a set of **keys** (what we look through), and associated **values** (what we retrieve), attention computes a weighted sum of values — where the weights are determined by the compatibility of the query with each key. It is the deep learning analogue of a fuzzy database query.

The impact was immediate and profound. Attention-augmented RNNs achieved state-of-the-art results in translation. The concept was then generalised and the recurrent components were discarded entirely in the landmark "Attention Is All You Need" paper (Vaswani et al., 2017), giving birth to the Transformer. Today,

attention is not merely a component — it is the primary computational primitive of the modern deep learning stack.

SECTION 02

Historical Background

YEAR	CONTRIBUTION	AUTHORS	SIGNIFICANCE
1990s	Encoder-Decoder RNNs	Sutskever et al.	Sequence-to-sequence learning; fixed-length bottleneck identified
2014	Seq2Seq with LSTM	Sutskever, Vinyals, Le	Deep encoder-decoder for machine translation
2015	Bahdanau Attention	Bahdanau, Cho, Bengio	First additive attention; aligned source tokens for translation
2015	Luong Attention	Luong, Pham, Manning	Multiplicative (dot-product) attention; global vs local variants
2016	Memory Networks	Weston et al.	Attention over external memory for Q&A tasks
2017	Transformer	Vaswani et al.	Attention-only architecture; multi-head self-attention; SOTA NMT
2018	BERT / GPT	Devlin et al. / OpenAI	Bidirectional / generative pre-training with transformers
2020+	Efficient Attention	Multiple groups	Sparse, linear, and Flash attention addressing $O(n^2)$ cost

Mathematical Foundations

Attention is fundamentally a function that maps a query and a set of key-value pairs to an output. All quantities are vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a *compatibility function* (also called a score function or alignment model) of the query with the corresponding key.

$$\text{Attention}(Q, K, V) = \text{softmax}(\text{score}(Q, K)) \cdot V$$

General attention formulation — the output is a weighted sum of values

Score Functions

The choice of score function is what distinguishes the various attention flavours:

NAME	FORMULA	NOTES
Dot-product	$\text{score} = QK^\top$	Fast; assumes Q and K have same dimension
Scaled dot-product	$\text{score} = QK^\top / \sqrt{d_k}$	Prevents softmax saturation in high dimensions
Additive (Bahdanau)	$\text{score} = v^\top \tanh(W_1Q + W_2K)$	Learnable MLP; works when Q and K differ in size
Bilinear	$\text{score} = QW K^\top$	Generalises dot-product; W is learned weight matrix
Cosine	$\text{score} = \frac{Q \cdot K}{\ Q\ \ K\ }$	Magnitude-invariant; bounded in $[-1, 1]$

The Softmax as a Distribution

After computing scores, the `softmax` function converts them into a probability distribution over positions. For a score vector $\mathbf{e}$ of length n :

$$\alpha_i = \frac{\exp(e_i)}{\sum_j \exp(e_j)}, \quad \sum_i \alpha_i = 1, \quad \alpha_i \geq 0$$

*Softmax over attention scores —
 α_i
is the attention weight for position
 i*

WHY SCALE BY $\sqrt{d_k}$?

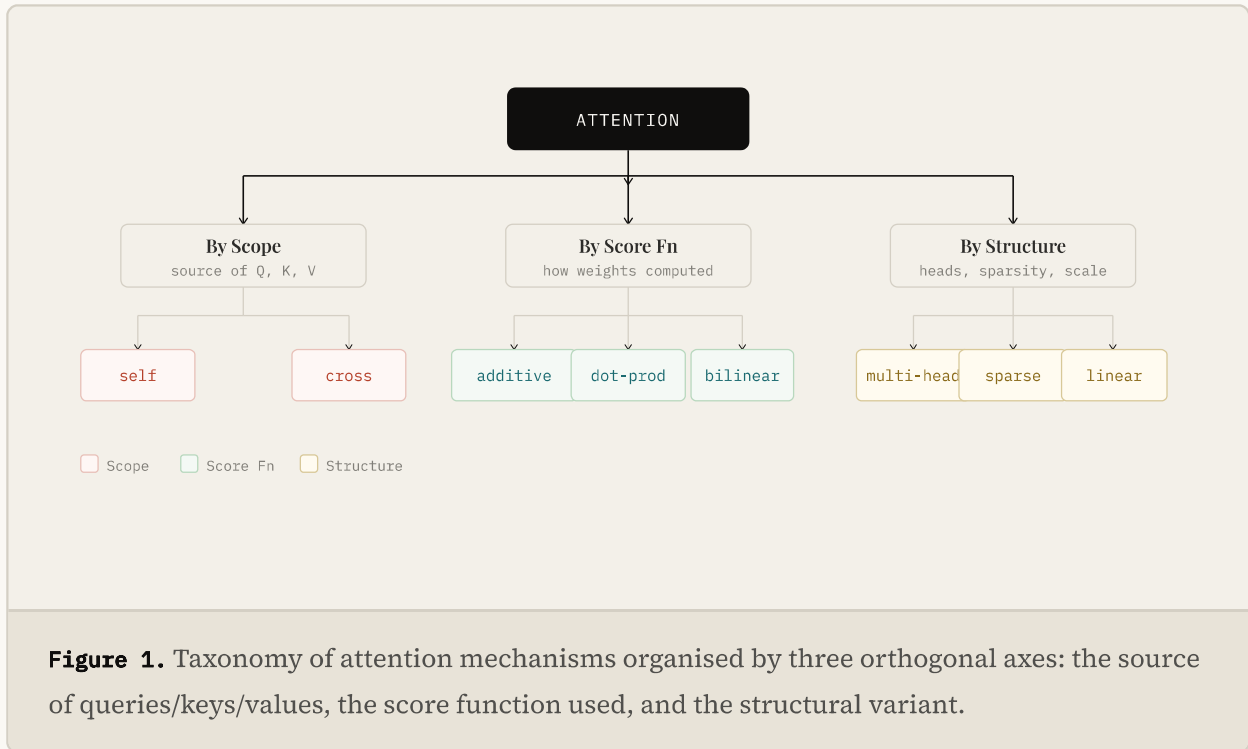
For large embedding dimensions d_k , dot products grow in magnitude roughly as d_k . This pushes the softmax into regions of extremely small gradients (the exponential saturates), making training unstable. Dividing by $\sqrt{d_k}$ keeps the expected dot-product magnitude close to 1 regardless of dimension, stabilising the gradient flow.

Attention as a Soft Dictionary Lookup

One powerful way to understand attention is to contrast it with a hard dictionary lookup. In a Python dictionary, a key either matches exactly or it doesn't.

Attention implements a *soft* version: every query matches every key to varying degrees, and the returned value is a smooth interpolation over all stored values weighted by those match scores. This makes the entire operation differentiable and trainable end-to-end via backpropagation.

Types of Attention Mechanisms



Scaled Dot-Product Attention

The canonical form introduced in the Transformer paper, and the default attention function in virtually all modern architectures, is **scaled dot-product attention**. Given queries $Q \in \mathbb{R}^{n \times d_k}$, keys $K \in \mathbb{R}^{m \times d_k}$, and values $V \in \mathbb{R}^{m \times d_v}$:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Scaled dot-product attention — the workhorse of modern deep learning

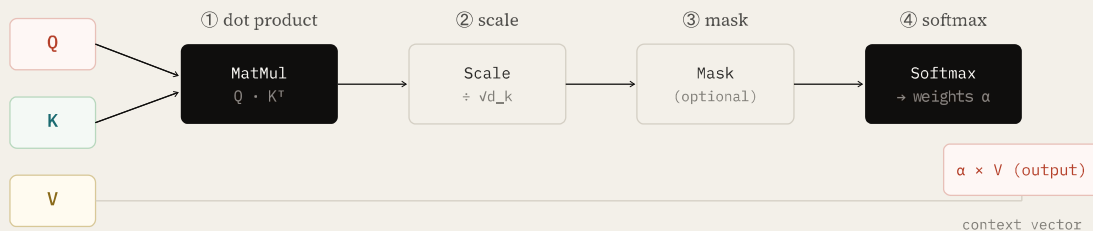


Figure 2. Computational graph of scaled dot-product attention. Q and K are used to compute alignment scores; these are scaled, optionally masked, and passed through softmax to obtain weights α which are then used to form a weighted sum of V .

Masking

Masking is an essential component in autoregressive (decoder) settings. The *causal mask* (also called a look-ahead mask) prevents position i from attending to positions $j > i$ by setting those attention logits to $-\infty$ before the softmax, which

forces their weight to exactly zero. This preserves the autoregressive property: the model cannot cheat by looking at future tokens during training.

PADDING MASK VS. CAUSAL MASK

Padding mask: applied in both encoder and decoder to prevent attention to padding tokens (positions with no real content). These positions are set to $-\infty$.

Causal mask: applied in the decoder only; a triangular mask ensuring each position can only attend to itself and prior positions. Both are applied additively to the raw score matrix before softmax.

.

Multi-Head Attention

A single attention function computes one weighted summary of the value vectors. But natural language has many types of relationships: syntactic dependencies, coreference, semantic similarity, positional proximity. **Multi-head attention** (MHA) runs h independent attention functions in parallel — each with its own learned linear projections of Q, K, and V — and concatenates their outputs.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Multi-head attention — h heads are run in parallel and concatenated

$$\text{head}_i = \text{Attention}\left(QW_Q^{(i)}, KW_K^{(i)}, VW_V^{(i)}\right)$$

Each head uses its own learned projection matrices

$$W_Q^{(i)}, W_K^{(i)}, W_V^{(i)} \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

The projection matrices are typically designed so that the total computation remains constant: if the model dimension is `d_model` and there are `h` heads, each head operates on dimension `d_k = d_model / h`. The concatenated output is projected back to `d_model` by $W_O \in \mathbb{R}^{(h \cdot d_v) \times d_{\text{model}}}$.

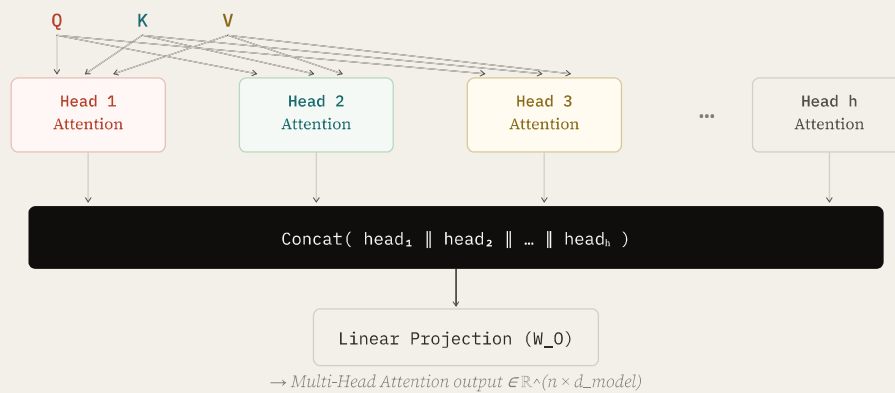


Figure 3. Multi-head attention: h attention heads run in parallel, each attending to different representation subspaces. Outputs are concatenated and linearly projected back to d_{model} .

WHY MULTIPLE HEADS?

Each attention head is free to specialise in a different type of relationship. Empirical analysis shows that different heads in trained transformers attend to different linguistic phenomena: some track subject-verb agreement, others resolve coreferences, others focus on adjacent token relationships. The multi-head design gives the model the expressive capacity to simultaneously model all these relationships within a single layer.

In practice, implementations batch the heads together for efficiency using a single large matrix multiplication followed by a reshape. The key computational insight is that computing h heads of dimension $d_k = d_{\text{model}}/h$ costs the same as one head of dimension d_{model} — but yields far richer representations.

Self-Attention & Cross-Attention

Self-Attention

In **self-attention**, the queries, keys, and values all come from the same sequence. Each token in the sequence attends to every other token in that same sequence. This allows the model to build contextualised representations — the embedding of a token is updated based on all other tokens in context, so the same word can have different representations in different contexts ("bank" next to "river" versus "bank" next to "deposit").

Self-Attention: $Q = XW_Q$, $K = XW_K$, $V = XW_V$ (same input X)



All three roles — query, key, value — are projected from the same input matrix X

Cross-Attention

In **cross-attention** (also called encoder-decoder attention), the queries come from one sequence (typically the decoder's current state) and the keys and values come from another sequence (typically the encoder's output). This is how the decoder learns to "look at" the source sentence while generating each target token.

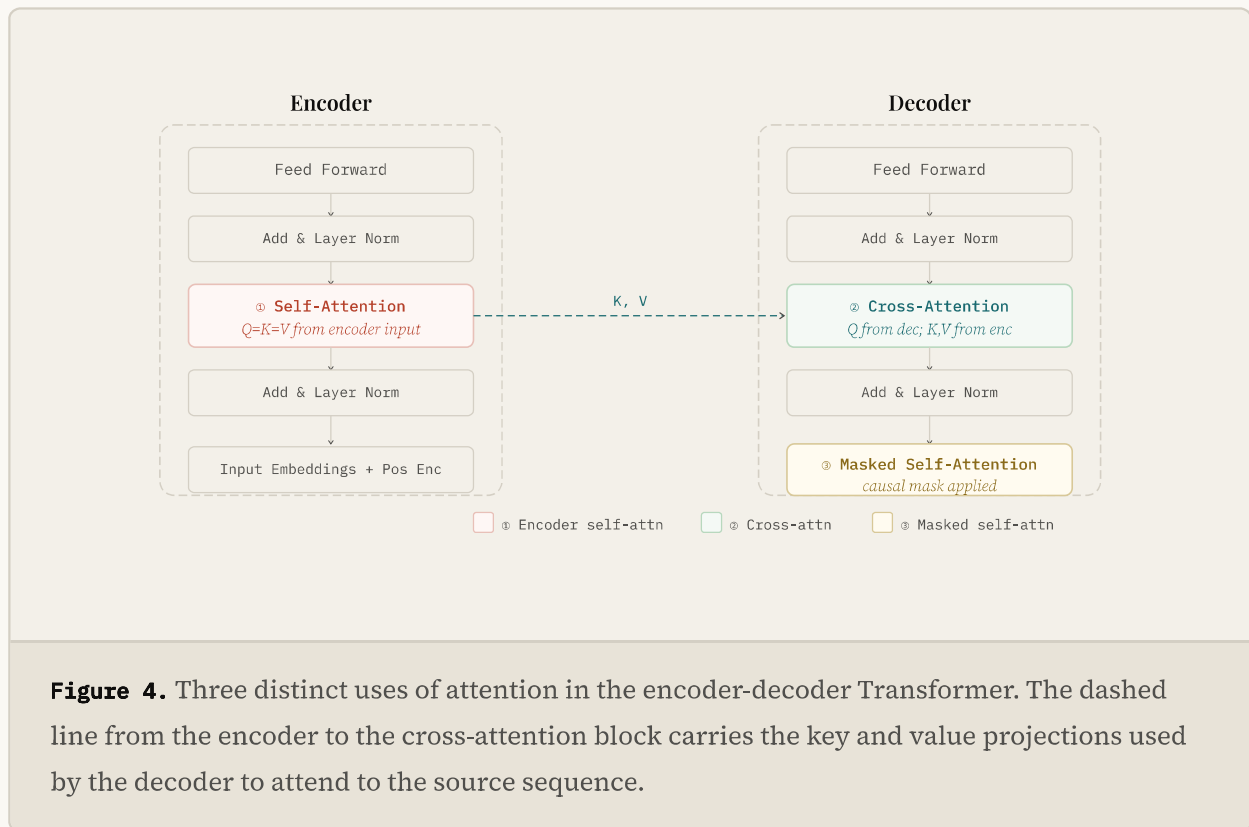
Cross-Attention: $Q = X_{\text{dec}} W_Q$, $K = X_{\text{enc}} W_K$, $V = X_{\text{enc}} W_V$

Queries from decoder; keys and values from encoder — the cross-modal bridge

PROPERTY	SELF-ATTENTION	CROSS-ATTENTION
Q source	Same sequence	Target/decoder sequence
K, V source	Same sequence	Source/encoder sequence
Primary use	Contextualisation within a sequence	Conditioning on a separate context
Used in	Encoder, Decoder (masked)	Decoder (between enc/dec layers)
Typical task	NLU, language modelling	Translation, captioning, ASR
.		

Attention in the Transformer Architecture

The Transformer uses attention in three distinct locations. Understanding these roles is essential for understanding how the full architecture works:



Each of these three attention types serves a distinct purpose. Encoder self-attention builds rich, bidirectional contextualised representations of the source. Masked decoder self-attention generates target embeddings causally, attending only to previously generated tokens. Cross-attention is the bridge: it allows the decoder to dynamically focus on the most relevant parts of the encoded source at every generation step.

Positional Encoding

Attention by itself is *permutation-invariant*: if you shuffle the input tokens, the attention weights shuffle accordingly and the outputs are equivalent (modulo reordering). This is desirable for some tasks but catastrophic for language, where word order carries meaning. **Positional encodings** inject position information into token embeddings before they enter the attention layers.

Sinusoidal Positional Encoding (Vaswani et al., 2017)

The original Transformer used deterministic sinusoidal encodings added to the token embeddings. For position pos and dimension i :

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

Even dimensions use sine

$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

Odd dimensions use cosine

Learned Positional Embeddings

BERT and GPT replaced sinusoidal encodings with simple learned embedding vectors — one per position, trained end-to-end like token embeddings. This works well in practice but is limited to sequences up to the training context length.

Rotary Position Embedding (RoPE)

More recent architectures (LLaMA, GPT-NeoX, PaLM) use **RoPE**, which encodes position by rotating query and key vectors in 2D planes before the dot-product. This elegantly encodes *relative* positions: the dot product between Q and K at positions m and n depends only on their difference $m - n$, enabling better generalisation to context lengths beyond those seen during training.

ALIBI — ATTENTION WITH LINEAR BIASES

An alternative approach (Press et al., 2021) adds a learnable linear bias to the attention scores based on the distance between query and key positions. Rather than modifying embeddings, it directly biases attention: distant tokens are penalised more, reducing their influence. This achieves strong extrapolation to longer sequences at essentially zero additional parameter cost.

.

Advanced Variants

VARIANT	KEY IDEA	COMPLEXITY	USED IN
Standard MHA	h parallel full-context attention heads	$O(n^2 \cdot d)$	BERT, GPT-2, T5
Grouped-Query Attention (GQA)	Multiple query heads share K and V heads; reduces KV cache	$O(n^2 \cdot d)$	LLaMA 2/3, Mistral
Multi-Query Attention (MQA)	All queries share single K and V projection; extreme KV compression	$O(n^2 \cdot d)$	PaLM, Falcon
Sparse Attention	Each query attends to a fixed subset of keys (local + strided)	$O(n \cdot \sqrt{n} \cdot d)$	GPT-3, Longformer
Linformer	Projects K and V to a fixed low-rank $n \times k$ matrix	$O(n \cdot k \cdot d)$	Long-sequence classification
Performer	FAVOR+ random feature map approximates softmax kernel	$O(n \cdot d)$	Long biological sequences
FlashAttention	IO-aware tiled computation; exact, not approximate; avoids $N \times N$ matrix	$O(n^2 \cdot d)$, fast I/O	GPT-4, LLaMA 3, Claude
Sliding Window Attention	Each token attends only to $\pm w/2$ neighbours; global tokens optional	$O(n \cdot w \cdot d)$	Longformer, BigBird

VARIANT	KEY IDEA	COMPLEXITY	USED IN
Cross-Layer Attention	KV from earlier transformer layers reused in later layers	$O(n^2 \cdot d)$	Research architectures

FlashAttention — A Closer Look

FlashAttention (Dao et al., 2022) does not approximate attention — it computes the exact same result as standard attention but reorganises the computation to avoid materialising the full $n \times n$ attention matrix in slow GPU HBM (high-bandwidth memory). By processing the computation in tiles that fit in on-chip SRAM, FlashAttention achieves 2–4× speed improvements and near-linear memory usage, making it the de facto standard for training and inference of large models.

FLASHATTENTION KEY INSIGHT

The bottleneck in standard attention is not the FLOPs — it is the memory bandwidth required to read and write the $n \times n$ score matrix. FlashAttention fuses the score computation, softmax, and weighted sum into a single kernel, reading each element of Q, K, V only once and never storing the full $n \times n$ matrix. The softmax is computed incrementally using the online softmax trick.

Complexity & Efficiency

The dominant computational cost of standard attention is the matrix multiplication $Q \cdot K^T$, which produces an $n \times n$ matrix for sequence length n . For long sequences this becomes prohibitive:

SEQUENCE LENGTH	ATTENTION MATRIX SIZE	MEMORY (FP16)	PRACTICAL?
512	512×512	~0.5 MB	✓ Trivial
2,048	$2K \times 2K$	~8 MB	✓ Standard
8,192	$8K \times 8K$	~128 MB	✓ With care
32,768	$32K \times 32K$	~2 GB	⚠ Challenging
131,072	$128K \times 128K$	~32 GB	✗ Infeasible (standard)

This $O(n^2)$ scaling is why efficient attention variants have become essential for long-context models. In practice, the field has moved toward combining FlashAttention (for IO efficiency) with architectural changes such as grouped-query attention (to reduce KV cache size during inference) and sliding-window attention (to reduce the effective context that must be attended to).

KV CACHE IN INFERENCE

During autoregressive generation, keys and values computed for previous positions are cached and reused. The *KV cache* grows linearly with sequence length and is often the binding memory constraint during inference. Grouped-query attention and multi-query attention reduce the KV cache size by sharing key/value projections across query heads, trading a small quality reduction for a significant reduction in inference memory and latency.

Applications

DOMAIN	TASK	ATTENTION ROLE	EXAMPLES
NLP	Language modelling, translation, summarisation	Self-attention + cross-attention	GPT-4, Claude, Gemini, T5
Vision	Image classification, detection, segmentation	Patch self-attention	ViT, DINO, SAM, CLIP
Multimodal	Image captioning, VQA, text-to-image	Cross-attention between modalities	Flamingo, DALL-E, Stable Diffusion
Audio	Speech recognition, TTS, music generation	Self-attention on spectrograms	Whisper, AudioLM, MusicLM
Code	Code completion, synthesis, review	Long-context self-attention	Codex, StarCoder, DeepSeek-Coder
Biology	Protein structure prediction	Residue-pair self-attention	AlphaFold2, ESMFold
Robotics	Policy learning, scene understanding	Cross-attention (sensors → actions)	RT-2, Perceiver IO

Implementation

Below is a complete, production-quality implementation of multi-head attention in PyTorch, closely matching the architecture of models like BERT and GPT.

```
PYTHON · PYTORCH

import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class ScaledDotProductAttention(nn.Module):
    """
    Computes scaled dot-product attention.
    Supports optional causal masking and dropout.
    """

    def __init__(self, dropout: float = 0.0):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

    def forward(
        self,
        q: torch.Tensor,          # (B, H, T_q, d_k)
        k: torch.Tensor,          # (B, H, T_k, d_k)
        v: torch.Tensor,          # (B, H, T_k, d_v)
        mask: torch.Tensor | None = None,
    ) -> tuple[torch.Tensor, torch.Tensor]:

        d_k = q.size(-1)
        scale = math.sqrt(d_k)

        # ① Compute raw attention scores: (B, H, T_q, T_k)
        scores = torch.matmul(q, k.transpose(-2, -1)) / scale

        # ② Apply mask (set -inf where mask is True)
        if mask is not None:
            scores = scores.masked_fill(mask, float('-inf'))
```

```

        # ③ Softmax over key dimension
        attn_weights = F.softmax(scores, dim=-1)    # (B, H, T_q, T_k)
        attn_weights = self.dropout(attn_weights)

        # ④ Weighted sum of values
        output = torch.matmul(attn_weights, v)      # (B, H, T_q, d_v)
        return output, attn_weights

class MultiHeadAttention(nn.Module):
    """
    Multi-head attention as in 'Attention Is All You Need'.
    Supports self-attention and cross-attention.

    Args:
        d_model: Total model embedding dimension.
        n_heads: Number of attention heads.
        dropout: Dropout applied to attention weights.
        bias: Whether to add bias to linear projections.
    """

    def __init__(
        self,
        d_model: int = 512,
        n_heads: int = 8,
        dropout: float = 0.1,
        bias: bool = True,
    ):
        super().__init__()
        assert d_model % n_heads == 0, "d_model must be divisible by n_heads"

        self.d_model = d_model
        self.n_heads = n_heads
        self.d_k = d_model // n_heads    # per-head key dimension
        self.d_v = d_model // n_heads    # per-head value dimension

        # Learned linear projections for Q, K, V and the output
        self.W_q = nn.Linear(d_model, d_model, bias=bias)
        self.W_k = nn.Linear(d_model, d_model, bias=bias)
        self.W_v = nn.Linear(d_model, d_model, bias=bias)
        self.W_o = nn.Linear(d_model, d_model, bias=bias)

        self.attention = ScaledDotProductAttention(dropout)
        self.dropout = nn.Dropout(dropout)

```

```

def split_heads(self, x: torch.Tensor) -> torch.Tensor:
    """Reshape (B, T, d_model) → (B, H, T, d_k) for parallel head computation
    B, T, _ = x.size()
    return x.view(B, T, self.n_heads, self.d_k).transpose(1, 2)

```

```

def merge_heads(self, x: torch.Tensor) -> torch.Tensor:
    """Reshape (B, H, T, d_k) → (B, T, d_model) to collapse heads."""
    B, H, T, d_k = x.size()
    return x.transpose(1, 2).contiguous().view(B, T, H * d_k)

```

```

def forward(
    self,
    query: torch.Tensor,                # (B, T_q, d_model)
    key: torch.Tensor,                  # (B, T_k, d_model)
    value: torch.Tensor,                # (B, T_k, d_model)
    mask: torch.Tensor | None = None,
) -> tuple[torch.Tensor, torch.Tensor]:
    """

```

For self-attention: pass the same tensor for query, key, value.
 For cross-attention: pass decoder state as query, encoder output
 as key and value.

Returns:

output: (B, T_q, d_model)
 attn_weights: (B, H, T_q, T_k) – useful for analysis/viz

"""

Project and split into H heads

Q = self.split_heads(self.W_q(query))

K = self.split_heads(self.W_k(key))

V = self.split_heads(self.W_v(value))

Per-head attention

attn_out, attn_weights = self.attention(Q, K, V, mask=mask)

Merge heads and project to d_model

output = self.W_o(self.merge_heads(attn_out))

return output, attn_weights

```

def make_causal_mask(T: int, device: torch.device) -> torch.Tensor:
    """Upper-triangular boolean mask – True where attention is forbidden.
    Prevents position i from attending to positions j > i.
    """
    return torch.triu(torch.ones(T, T, dtype=torch.bool, device=device), diag

```



```

# — Example usage —
if __name__ == "__main__":
    B, T, d = 2, 16, 512    # batch=2, seq_len=16, d_model=512

    mha = MultiHeadAttention(d_model=d, n_heads=8, dropout=0.1)
    x = torch.randn(B, T, d)

    # Self-attention with causal mask (decoder setting)
    mask = make_causal_mask(T, device=x.device)
    out, w = mha(x, x, x, mask=mask.unsqueeze(0).unsqueeze(0))

    print(f"Output: {out.shape}")      # → torch.Size([2, 16, 512])
    print(f"Weights: {w.shape}")      # → torch.Size([2, 8, 16, 16])

```

The `split_heads` / `merge_heads` operations are the key implementation trick: by reshaping the Q, K, V tensors before the attention computation, all heads are computed with a single batched matrix multiplication, achieving the same result as running h independent attention functions but with no additional computational overhead.

Uditya Narayan Tiwari

Developer & Technical Content Creator

AI/LLM Tooling · Model Context Protocol ·

Python

[Portfolio ↗](#)

[GitHub ↗](#)

[LinkedIn ↗](#)

[Knowledge Base ↗](#)

This report provides a comprehensive technical examination of the attention mechanism in deep learning. All mathematical formulations and architectural descriptions reference the original published literature. The PyTorch implementation is provided for educational purposes. © 2026 Uditya Narayan Tiwari.